

CSCE 735 PARALLEL COMPUTING · MODULE 04 · LECTURE NOTES

Synchronisation and Memory Models

What 'happens before' means when nothing is sequential.

MODULE OUTCOMES

- Use mutexes and condition variables correctly.
- Explain why a memory model is necessary.
- Use atomics with appropriate memory ordering.
- Explain sequential consistency and why hardware does not provide it.
- Recognise and avoid the common synchronisation errors.

1 Locks

Primitive	Provides	Cost and caveats
Mutex	Mutual exclusion over a critical section.	Roughly 20–100 ns uncontended; under contention, far more , because threads block and the operating system is involved.
Reader-writer lock	Many concurrent readers, or one exclusive writer.	More expensive than a plain mutex when uncontended , because the bookkeeping is heavier. It only pays when reads genuinely dominate and are long enough to matter — which is less often than people assume when reaching for it.
Condition variable	Wait efficiently until a predicate becomes true.	Requires an associated mutex, and must be waited on in a loop — see below.
Atomic	An indivisible read-modify-write on one word.	The cheapest primitive , and limited to word-sized operations. §3.
Barrier	All participating threads reach a point before any proceeds.	Costs the slowest thread's time, multiplied by the thread count — which is why load imbalance is so expensive (Module 06).
Semaphore	A counted resource.	General and correspondingly easy to misuse. Prefer a more specific primitive when one fits.

```
std::unique_lock<std::mutex> lk(m);
while (!ready)           // WHILE, not IF
    cv.wait(lk);

// or, preferably, the predicate overload which writes the loop for you:
cv.wait(lk, []{ return ready; });
```

THE LOOP IS NOT OPTIONAL

Spurious wakeups are explicitly permitted by the standard. `wait()` may return without any corresponding notify having occurred, because allowing this makes the implementation substantially simpler and faster on some platforms.

And even without spurious wakeups, the predicate may have been consumed. Between the notifying thread's notify and this thread reacquiring the mutex, another waiting thread may have woken first and taken the item.

With if rather than while, the code proceeds with the predicate false — dequeuing from an empty queue, or using data that is not ready.

This is the single most common condition variable bug, and the predicate overload of `wait()` exists specifically to make it hard to write. Use it.

LOCK ORDERING PREVENTS DEADLOCK

Deadlock requires a cycle in the wait-for graph. Thread A holds lock 1 and waits for lock 2; thread B holds lock 2 and waits for lock 1. Neither can proceed.

A global acquisition order makes a cycle impossible. If every thread always acquires locks in the same defined order — by address, by a documented hierarchy, by any consistent rule — then a thread holding a lower-ordered lock never waits for a higher-ordered one held by a thread waiting on a lower, and no cycle can form.

So define the order, document it where the locks are declared, and never violate it. `std::scoped_lock` acquires several locks at once using a deadlock-avoidance algorithm, which handles the cases where a fixed order is awkward.

And hold locks for as short a time as possible. Never across I/O, which has unbounded duration; never across a callback into code you do not control, which may re-enter and attempt to acquire the same lock. Both are common sources of deadlock in otherwise careful code.

2 Memory models

NOTHING HAPPENS IN THE ORDER YOU WROTE IT

The compiler reorders memory operations. It hoists loads out of loops, sinks stores past branches, merges adjacent accesses, and eliminates operations it can prove are redundant.

The processor reorders memory operations. Out-of-order execution issues instructions as their operands become available, and store buffers allow a write to be committed to the processor's own view long before it becomes visible to others.

Both are entirely legal, because both preserve the semantics of a *single-threaded* program. That is the only guarantee the hardware ever offered, and it is sufficient for the overwhelming majority of code.

So without synchronisation, another thread may observe your writes in a different order than you performed them — and may observe some and not others. **A memory model is the contract specifying exactly when that may not happen**, which is why a language needs one before it can support threads at all.

Ordering	Guarantee	When to use
seq_cst (sequential consistency)	A single total order of all sequentially consistent operations, which every thread agrees on.	The default, and the right default. Strongest and simplest to reason about. The most expensive on weakly ordered hardware, and nearly free on x86.
acquire / release	A release store synchronises-with an acquire load of the same variable: everything sequenced before the release becomes visible to everything sequenced after the acquire. A one-way barrier in each direction.	Sufficient for most lock-free data structures , and measurably cheaper than seq_cst on ARM and POWER. This is what a mutex does internally.
relaxed	Atomicity only — the operation is indivisible. No ordering guarantees whatsoever.	Counters that nobody reads until a later synchronisation point; statistics; reference counts on increment. Easy to misuse.

SEQUENTIAL CONSISTENCY IS WHAT YOU ASSUME AND NOT WHAT YOU GET

Sequential consistency means there exists a single global ordering of all memory operations, consistent with each thread's program order. It is the model everyone reasons with intuitively, and almost everyone assumes without realising they are assuming it.

No mainstream processor provides it by default, because it would forbid store buffering and most memory-level parallelism, at a large performance cost.

x86 is comparatively strong — total store order, which permits only store-load reordering. **ARM and POWER are weak** and reorder far more freely.

So code that works correctly on x86 can fail on ARM, which is now a mainstream target in servers, laptops, and phones — and the failure appears as an impossible result, not a crash. **Reason using the language's memory model rather than your mental model of a particular processor**, and the code is portable by construction.

3 Atomics in practice

- **Counters and flags.** A single word updated by many threads, where taking a mutex would be absurd overhead relative to the operation.
- **Compare-and-swap loops** for small lock-free updates (Module 05) — read the current value, compute the new one, and swap it in only if nothing changed meanwhile.
- **And they are not free.** An atomic read-modify-write acquires exclusive ownership of the cache line, so concurrent atomics on the same address serialise at the coherence layer. **Under heavy contention an atomic counter can be slower than a mutex**, because the mutex at least lets a thread sleep instead of spinning on a line it keeps losing.
- **Contended atomics serialise exactly as locks do**, with the additional cost of repeated failed CAS attempts whose work is discarded. **The primitive is cheap; the contention is not** — and it is the contention that costs.
- **So per-thread accumulation beats both** whenever the operation is associative and the combination can be deferred, which covers most reductions and most counters. This is the same conclusion as Module 03's reduction clause, reached from the hardware side.

THE MISTAKES THAT RECUR

Double-checked locking without atomics. Checking a flag, taking a lock, checking again, and initialising. **The classic broken idiom** — without proper memory ordering, another thread can see the flag set before the initialisation it guards is visible. Correct only with acquire-release or seq_cst atomics, and `std::call_once` or a function-local static is simpler and correct by construction.

if instead of while on a condition variable (§1).

Holding a lock across I/O or across a callback. I/O has unbounded duration, so the critical section does too; a callback may re-enter and attempt to acquire the same lock, which deadlocks immediately with a non-recursive mutex.

Assuming volatile means atomic. It does not, in C or C++. It prevents certain compiler optimisations — which is why it works for memory-mapped hardware registers — and says nothing whatsoever about processor reordering or about atomicity. Java and C# give it different meanings, which is a substantial source of the confusion.

And lock-free code written without reference to the memory model, which works on the machine it was tested on and fails when the architecture changes.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Herlihy & Shavit — The Art of Multiprocessor Programming	The reference for this module and Module 05. Library copy; the authors' slides are free and cover the core material.
Preshing on Programming — memory ordering articles (free)	The clearest free explanation of acquire-release and memory ordering , with diagrams and runnable examples. Start here if §2 is unfamiliar.
Sutter — atomic Weapons (free talks)	The C++ memory model in two talks, by someone who helped specify it.
Sorin, Hill & Wood — A Primer on Memory Consistency and Cache Coherence (free)	The hardware side of §2, rigorously. Free through many institutions.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Measure uncontended and contended mutex cost on your machine, as a function of thread count.
- 2 Measure a reader-writer lock against a mutex at several read/write ratios. Find the ratio at which it starts to win.
- 3 Write a condition variable wait with if and construct a case where it fails. Then fix it.
- 4 Construct a deadlock with two locks acquired in opposite orders, then fix it with a defined order and with `std::scoped_lock`.
- 5 **Write a store-buffer litmus test** — two threads, each storing to one variable and loading the other — and run it millions of times. Confirm that both loads returning zero is observable on x86.
- 6 Run the same test with `seq_cst` atomics and confirm the outcome disappears.
- 7 Measure the cost of `seq_cst` against acquire-release on your hardware. Compare x86 and ARM if you have access to both.
- 8 Implement an atomic counter and a per-thread accumulator. Measure both at 1, 4, and 32 threads and plot the crossover.
- 9 Implement double-checked locking incorrectly, then correctly, then replace it with a function-local static.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name six synchronisation primitives with their costs, and say when a reader-writer lock pays.
- 2 Why must a condition variable be waited on in a loop? Give two reasons.
- 3 What does deadlock require, and what single rule prevents it?

- 4 Why do the compiler and processor reorder memory operations, and why is it legal?
- 5 Describe the three C++ memory orderings and when each is appropriate.
- 6 What is sequential consistency, and why does hardware not provide it?
- 7 Why are contended atomics not cheap, and what beats them?
- 8 Name five recurring synchronisation mistakes.
- 9 What does volatile actually mean in C++?